SI100B Fall 2026 · Recitation 6B

PA2 讲解 · PA3 指导 · Lec11/12 · Midterm Review

Azuma Tang | 2026-10-30

今日内容

1. PA2 详细讲解
2. Lec11 & Lec12 示例
3. PA3 起步指导
4. Midterm 全量复习
5. Notebook & 答疑

PA2 作业讲解

PA2 · 总览 | 四道题分别在练什么

一条主线

- T1 纯函数：给定参数，返回结果；不打印、不改外部变量。
- T2 列表：只读、造新列表、原地修改，三种副作用要分清。
- T3 状态机：队列、时间、完成记录、快照，先摆状态再写命令。
- T4 记录系统：tuple、切片、排序、交易历史，重点是“当前顺序”。
- 拿到题先问：函数返回什么？有没有改传入对象？状态什么时候保存？

拿到题先写这三行

1. 返回值是什么？
2. 会不会修改传入对象？
3. 输出/快照在什么时候发生？

Source: PA2_Solution/T1–T4

PA2 · T1 | T1：先定“返回什么”，再动循环

题目要什么

- is_even(n)：返回 n 是否为偶数，返回 bool，不打印。
- larger(x, y)：返回较大的数；相等时返回谁都行，但每条路径都要 return。
- sum_odd(a, b)：闭区间内所有奇数的和。
- count_divisible(a, b, d)：闭区间内能被 d 整除的个数。
- 闭区间用 range(a, b + 1)；漏掉 +1 是最常见的丢分点。

先手推，不急着写

```
sum_odd(1, 5) -> 1 + 3 + 5 = 9
count_divisible(1, 10, 3) -> 3, 6, 9 -> 3
sum_odd(4, 4) -> 0
is_even(-7) -> False  # -7 % 2 == 1
```

Source: PA2_Solution/T1.py, lines 4–32

PA2 · T1 | T1：四个函数的代码骨架

逐段看

- is_even 直接返回布尔表达式，不必再写 if/else。
- larger 先处理 x > y，剩下那条路径再 return y。
- sum_odd 先把 total 置 0，再在循环里筛奇数。
- count_divisible 把“计数”与“求和”分开，cnt 只加 1。
- return 放在循环结束后；不要在第一次迭代就返回。

参考实现

```
def is_even(n):
    return n % 2 == 0

def larger(x, y):
    if x > y:
        return x
    return y

total = 0
for k in range(a, b + 1):
    if k % 2 == 1:
        total += k
return total
```

Source: PA2_Solution/T1.py, lines 4–32

PA2 · T1 | T1：易错点与测试

返回值的坑

- 用 print 顶替 return：屏幕有数，函数其实返回 None。
- larger 只写 if 分支：x <= y 时函数走到底，返回 None。
- sum_odd 在循环里 return：只算了第一个奇数。

循环的坑

- range(a, b) 少写 +1：b 是奇数时正好漏掉最后一个。
- total 写在循环里：每轮从 0 开始，最后只剩最后一个奇数。
- k % 2 直接当条件能跑，但写 == 1 更清楚。

边界测试

- sum_odd(4, 4) == 0，空区间不报错。
- sum_odd(5, 1) == 0，start > stop 时循环不执行。
- count_divisible(1, 3, 5) == 0，除数比区间还大。

Source: PA2_Solution/T1.py; Recitation 5 PDF pp. 4–8

PA2 · T2 | T2：查询和累加都是只读，别顺手改列表

题目要什么

- get_stock(stock, index)：原样返回那个位置的数量。
- total_stock(stock)：从头到尾累加。
- 两个都是只读函数：调用前后 stock 必须完全一样。
- 空列表也要能算：total_stock([]) 是 0，不是异常。
- 参数传进来的是列表引用；函数里改元素，外面看得见。

先手推

```
stock = [3, 0, 5]
get_stock(stock, 1) -> 0
total_stock(stock) -> 3 + 0 + 5 = 8
调用后 stock 仍是 [3, 0, 5]
```

Source: PA2_Solution/T2.py, lines 4–14

PA2 · T2 | T2：low_stock 造新表，restock 改原表

题目要什么

- low_stock：返回库存低于 threshold 的下标，严格小于，等于不算。
- restock：每个库存加 amount，但不超过 limit；原地改，返回 None。
- 一个交出新列表，一个修改传入列表，这是 T2 的分水岭。
- low_stock([3,0,5], 3) 返回 [1]，不是 [0]。
- restock([2,9,4], 5, 10) 得到 [7, 10, 9]。

代码骨架

```
# low_stock
res = []
for i in range(len(stock)):
    if stock[i] < threshold:
        res.append(i)
return res

# restock
for i in range(len(stock)):
    stock[i] = min(stock[i] + amount, limit)
```

Source: PA2_Solution/T2.py, lines 17–33

PA2 · T2 | T2：新列表还是原列表?

low_stock

- 返回的是下标，不是库存数量。
- res 是新的 list，原 stock 不变。
- 可以用 enumerate 写，但循环 + append 更直观。

restock

- for inv in stock 里 inv += amount 改不到原表。
- 必须用 stock[i] = ... 按索引写回。
- 函数没有 return，调用处别写 s = restock(...)。

测试

- 只读：before = stock[:]，调用后 assert stock == before。
- 空表：total_stock([]) == 0。
- 封顶：restock([0,0], 3, 2) 得到 [2, 2]。

Source: PA2_Solution/T2.py; Recitation 5 PDF pp. 4–8

PA2 · T3 | T3：先把状态摆出来，再谈 MOVE

四个状态

- queues：每台打印机一个队列；任务是 (job_id, pages)。
- time：当前秒数；completed：完成记录；snapshots：快照。
- 前两个是“现在”，后两个是“留痕”，不要混着改。
- MOVE：先跨队列找到并 pop，再插入目标队列的 position。
- position 按“移除之后”的队列算；可以等于长度，表示追加到末尾。

take_job：找到并移除

```
def take_job(queues, job_id):
    for queue in queues:
        for position in range(len(queue)):
            if queue[position][0] == job_id:
                return queue.pop(position)

job = take_job(queues, job_id)
queues[printer].insert(position, job)
```

Source: PA2_Solution/T3.py, lines 4–8, 23–30

PA2 · T3 | T3：TICK 一秒一格，每台只动队首

TICK 的顺序

- TICK n 推进 n 秒；每一秒所有打印机同时工作。
- 每台每秒只处理队首一个任务、只减 1 页。
- pages == 1 当秒完成，记录 (time, job_id, printer)。
- 否则用新 tuple 替换队首，页数减 1。
- 同一秒不能接着处理下一个任务；空闲打印机跳过，但秒照样走。

手推 TICK 2

```
秒末 1  打印机0: [1:1页, 2:1页]  打印机1: []  完成 3
秒末 2  打印机0: [2:1页]         打印机1: []  完成 1

# 任务 2 要等第 3 秒才开始
for _ in range(command[1]):
    time += 1
    for printer in range(n):
        queue = queues[printer]
        if not queue:
            continue
        job_id, pages = queue[0]
        if pages == 1:
            queue.pop(0)
            completed.append(
                    (time, job_id, printer))
        else:
            queue[0] = (job_id, pages - 1)
```

Source: PA2_Solution/T3.py, lines 35–47; Recitation 5 PDF p. 6

PA2 · T3 | T3：CANCEL 不记完成，SNAP 要复制对层

为什么 queue[:] 够用

- 任务是 tuple，不可变；减页是换一个新 tuple，旧 tuple 还在。
- 所以只要把每支队列这个 list 复制一份，任务对象共享是安全的。
- 外层也必须新建：snapshots.append((time, [q[:] for q in queues]))。
- CANCEL 只是拿走任务，不写 completed。
- 如果任务用 list 存，queue[:] 只复制外层，要逐任务复制或 deepcopy。

快照代码

```
saved_queues = []
for queue in queues:
    saved_queues.append(queue[:])
snapshots.append((time, saved_queues))

# 如果任务本身是 list：
# from copy import deepcopy
# snapshots.append((time, deepcopy(queues)))
```

Source: PA2_Solution/T3.py, lines 49–54; T3_2.py, line 57

PA2 · T3 | T3：最容易错的五个地方

MOVE

- 先 insert 再 pop：任务短暂出现两份，位置也错。
- 同队列移动忘了先移除：position 按旧队列数，偏一位。
- position 等于长度表示追加到末尾，不是越界。

TICK

- 用 while 一次清空队列：多个任务被算成同一秒完成。
- time 加在循环外：TICK 3 只加了 1。
- 先减页再判断 == 1：1 页变 0 页，任务永远完不成。

SNAP / CANCEL

- 存 queues 引用：后续 TICK 会把快照一起改掉。
- CANCEL 也写 completed：DONE 数量对不上。
- SNAP 记 0 而不是当前 time：时间线错位。

Source: PA2_Solution/T3.py; Recitation 5 PDF pp. 5–8

PA2 · T4 | T4：tuple 不能改，只能整条换

题目要什么

- 商品是 (code, name, price, stock)，销售记录是 (code, quantity, cost)。
- buy 库存够就扣、记账、返回 cost；不够返回 None，且什么都不改。
- 失败不动库存、不动历史，这是判分最爱查的地方。
- 销售记录存成交价，后面调价不会回头改它。
- parse_item 把 code,name,price,stock 拆开并转类型。

buy 的关键路径

```
index = find_item_index(items, code)
item_code, name, price, stock = items[index]
if stock < quantity:
    return None
cost = price * quantity
items[index] = (
    item_code, name, price, stock - quantity)
sales.append((code, quantity, cost))
return cost
```

Source: PA2_Solution/T4.py, lines 29–45

PA2 · T4 | T4：切片切的是“当前顺序”

adjust / inventory

- start:end:step 与 list 切片完全一样：含 start、不含 end。
- 切的是当前 items 的顺序；前面 reorder 过，就按重排后的算。
- adjust 直接改原列表，不返回新列表。
- inventory_codes 取 code 列表；inventory_value 算 price * stock。
- 空切片：codes 返回 []，value 返回 None。

切片 + 调价

```
indices = range(len(items))[start:end:step]
for i in indices:
    code, name, old_price, stock = items[i]
    if rule == "discount":
        new_price = old_price * (1 - value / 100)
    elif rule == "markup":
        new_price = old_price * (1 + value / 100)
    else:
        new_price = value
    items[i] = (code, name, new_price, stock)
```

Source: PA2_Solution/T4.py, lines 59–80, 126–146

PA2 · T4 | T4：排序 key 管住并列，降序只降数值

二元组 key

- 并列按 code 升序，所以 key 返回 (数值, code)。
- 价格升序：(price, code)；价格降序：(-price, code)。
- 库存同理：(stock, code) 和 (-stock, code)。
- 不要直接 reverse=True：code 也会被倒序。
- sort 原地改并返回 None；sorted 返回新列表。

四个 key 函数

```
def price_asc_key(item):
    return item[2], item[0]

def price_desc_key(item):
    return -item[2], item[0]

def stock_asc_key(item):
    return item[3], item[0]

def stock_desc_key(item):
    return -item[3], item[0]
```

Source: PA2_Solution/T4.py, lines 85–104

PA2 · T4 | T4：历史记录与输出，别用当前价格重算收入

销售历史

- 成功销售才追加 (code, quantity, cost)；失败购买不写。
- sold 累加数量；revenue 累加成交时 cost。
- 后续调价不改历史 cost，否则收入会被改写。
- 空查询、两位小数和提示文字都要按题面输出。
- 测试：失败前后 items 相等、len(sales) 不变。

历史查询

```
def units_sold(sales, code):
    total = 0
    for item_code, quantity, _ in sales:
        if item_code == code:
            total += quantity
    return total

def total_revenue(sales):
    total = 0.0
    for _, _, cost in sales:
        total += cost
    return total
```

Source: PA2_Solution/T4.py, lines 149–165

PA2 · Notebook｜配套 Notebook：把四题拆成能单独跑的函数

PA2

- T1–T4 参考实现。
- 每题都有 assert 测试。
- 包含 MOVE / TICK / SNAP 手推例子。

Lec11 / Lec12

- Coordinate、Fraction 可运行示例。
- Workout / RunWorkout 继承示例。
- dunder、super、类变量与方法查找。

Midterm 易错诊所

- alias / 浅复制 / 深复制。
- sort 与 sorted、遍历时删除。
- 作用域、异常、字典、递归。

文件：Recitation6_B_Code_Examples.ipynb · 24 个代码单元全部执行通过

Lec11 & Lec12

Lec11 · OOP | Lec11：类与实例——class 定义类型

讲解

- 每个值都是对象：有类型、有数据、有能对它做的操作；1234 是 int 的实例。
- class 定义类型，不创建对象；Coordinate(3, 4) 才真的创建一个实例。
- __init__ 在实例创建后自动执行；self 就是刚创建的那个实例，Python 自动传入。
- self.x = x 才把值存到实例上；少写 self.，x 只是局部变量，函数结束就没了。
- 易错：__init__ 漏 self 会报参数多一个；实例属性不在类上，Coordinate.x 会 AttributeError。

代码与输出

```
class Coordinate:
    def __init__(self, x, y):
        self.x = x          # 存到实例上
        self.y = y

c = Coordinate(3, 4)
origin = Coordinate(0, 0)
print(c.x, c.y, origin.x)
print(type(c) is Coordinate)
# -> 3 4 0
# -> True
```

Lec11 · OOP｜Lec11：方法——第一个参数永远是实例自己

讲解

- 方法就是写在类里的函数，第一个参数固定是 self；调用时只传 other。
- c.distance(origin) 实际执行 Coordinate.distance(c, origin)：点号左边当 self。
- 点号既能取数据属性 c.x，也能取方法 c.distance，区别在有没有 ()。
- 方法里访问另一个对象要写 other.x，直接写 x 是未定义的名字。
- 易错：调用时又手动传 self 会报参数多一个；方法体写 x - other.x 会 NameError。

代码与输出

```
class Coordinate:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def distance(self, other):
        dx = self.x - other.x
        dy = self.y - other.y
        return (dx ** 2 + dy ** 2) ** 0.5

c = Coordinate(3, 4)
origin = Coordinate(0, 0)
print(c.distance(origin))          # 5.0
print(Coordinate.distance(c, origin))  # 5.0
```

Lec11 · OOP｜Lec11：__str__——print 看到的是你写的字符串

讲解

- 不定义 __str__，print(c) 得到 <... object at 0x...>，地址每次都变。
- print(obj) 等价于 print(str(obj))，str() 会找 __str__，它必须返回字符串。
- f-string 里的 f"{c}" 也走 __str__；repr(c) 走 __repr__，分工不同。
- 格式自己定：<3,4> 好读，也方便在输出里对齐。
- 易错：__str__ 里只 print 不 return 会得到 None；return self.x 返回 int 会 TypeError。

代码与输出

```
class RawPoint:
    def __init__(self, x, y):
        self.x, self.y = x, y

print(RawPoint(3, 4))
# <__main__.RawPoint object at 0x...>

class Coordinate:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __str__(self):
        return f"<{self.x},{self.y}>"

print(Coordinate(3, 4))   # <3,4>
print(f"{Coordinate(3, 4)}")  # <3,4>
```

Source: Lecture 11 · Print Representation

Lec11 · OOP｜Lec11： __mul__——a * b 背后是一次方法调用

讲解

- *、+、==、len()、print() 都是语法糖，背后各有一个 dunder 方法。
- 运算符方法要返回新对象，不要改 self，这样 a * b * c 才能接着算。
- a * b、a.__mul__(b)、Fraction.__mul__(a, b) 三种写法等价。
- 返回 float 也能打印一次，但类型已经被换掉，后面取 .num 就崩。
- 易错：return top / bottom 后 (a*b).num 会 AttributeError；在 __mul__ 里改 self 会把 a 改坏。

代码与输出

```
class Fraction:
    def __init__(self, num, denom):
        self.num, self.denom = num, denom
    def __str__(self):
        return f"{self.num}/{self.denom}"
    def __mul__(self, other):
        return Fraction(
            self.num * other.num,
            self.denom * other.denom)

a, b = Fraction(1, 4), Fraction(3, 4)
c = a * b
print(c, a)        # 3/16 1/4
print(c.num, c.denom)  # 3 16
```

Lec11 · OOP | Lec11： __float__ 与 isinstance

讲解

- float(c) 会找 __float__；int()、len()、bool() 也一样。
- 想让 Fraction 参与浮点运算，就实现 __float__，返回 num / denom。
- type(x) == Fraction 只认“恰好是”；isinstance 把子类也算进来。
- 写通用代码优先 isinstance；要拒绝别的类型时，判断完再 raise TypeError。
- 易错：isinstance(Fraction, c) 参数写反永远 False；type(x)==Fraction 判子类会漏。

代码与输出

```
class Fraction:
    def __init__(self, num, denom):
        self.num, self.denom = num, denom
    def __float__(self):
        return self.num / self.denom

class NegativeFraction(Fraction):
    pass

c = Fraction(3, 16)
n = NegativeFraction(1, 4)
print(float(c))                    # 0.1875
print(isinstance(c, Fraction))     # True
print(isinstance(n, Fraction))     # True
print(type(n) == Fraction)         # False
```

Source: Lecture 11 · Cast to float & Type Checking

Lec11 · OOP｜Lec11：返回新对象，还是原地修改

讲解

- 先读方法名和 docstring：get_inverse() 返回倒数，invert() 改自己、返回 None。
- 原地修改的方法没有返回值，x = f.invert() 会让 x 变成 None。
- reduce() 返回新的 Fraction，原对象不动；想原地约分得改 self.num。
- 两种风格都行，但同一个类里要一致，调用方按 docstring 用。
- 易错：print(f.invert()) 得到 None；以为 g.reduce() 改了 g，其实 g 还是 2/12。

代码与输出

```
class Fraction:
    def __init__(self, num, denom):
        self.num, self.denom = num, denom
    def __str__(self):
        return f"{self.num}/{self.denom}"
    def get_inverse(self):
        return self.denom / self.num
    def invert(self):
        n, d = self.num, self.denom
        self.num, self.denom = d, n

f = Fraction(3, 4)
print(f.get_inverse(), f)  # 1.333... 3/4
print(f.invert(), f)       # None 4/3
```

Source: Lecture 11 · Fraction: Return vs Mutate

Lec12 · Inheritance | Lec12：getter/setter 接口

讲解

- Python 不拦你：w.calories = 999 能改，w.size = "tiny" 还能凭空加属性。
- getter/setter 的价值是接口稳定：内部换成 datetime，外部照旧调 get_start()。
- setter 里能顺手校验，calories < 0 抛 ValueError；直接赋值绕过了这层。
- 类内部用 self.start 没问题；碰别人的对象，才走 getter/setter。
- 易错：setter 写成 calories = calories 漏 self；以为 w.calories = -5 会被拦住。

代码与输出

```
class Workout:
    def __init__(self, start, end, cal=200):
        self.start, self.end = start, end
        self.calories = cal
    def get_calories(self):
        return self.calories
    def set_calories(self, c):
        if c < 0:
            raise ValueError("calories < 0")
        self.calories = c

w = Workout("9/30 1:35 PM", "9/30 2:05 PM")
w.set_calories(260)
print(w.get_calories())   # 260
w.calories = -5           # 绕过 setter
print(w.get_calories())   # -5
```

Source: Lecture 12 · Getters and Setters

Lec12 · Inheritance｜Lec12：类变量 vs 实例变量——存在谁身上

讲解

- 类变量写在 class 体里（cal_per_hr = 200），全类共享一份。
- 实例变量写在 __init__ 里（self.calories = ...），每个实例各一份。
- 读取时先找实例、再找类：Workout.cal_per_hr = 250 会改变所有实例。
- w.cal_per_hr = 300 只给 w 加同名属性，类和其他实例都不动。
- 易错：想改全类却写 w.cal_per_hr；拿可变对象当类变量，所有实例共享同一个 list。

代码与输出

```
class Workout:
    cal_per_hr = 200       # 类变量：全类共享
    def __init__(self, start, end, cal=200):
        self.start, self.end = start, end
        self.calories = cal
    def get_calories(self):
        return self.calories

w1 = Workout("9/30 1:35 PM", "9/30 2:05 PM")
w2 = Workout("9/30 4:35 PM", "9/30 5:05 PM")
print(Workout.cal_per_hr, w1.cal_per_hr)  # 200 200
Workout.cal_per_hr = 250
print(w1.cal_per_hr, w2.cal_per_hr)       # 250 250
w1.cal_per_hr = 300
print(w1.cal_per_hr, w2.cal_per_hr)       # 300 250
```

Source: Lecture 12 · Class Variables

Lec12 · Inheritance｜Lec12：继承与 super().__init__——父类初始化不会自己跑

讲解

- class RunWorkout(Workout) 继承父类的数据和方法，自己只写差异。
- 子类有自己的 __init__，父类的不会自动跑，必须 super().__init__。
- super() 是父类的代理：先建好 start/end/calories，再设 self.elev。
- 参数顺序不同，调用时用关键字更稳，父类改签名时也更容易发现。
- 易错：忘了 super().__init__ 或写成没括号的 super().__init__，都会缺父类属性。

代码与输出

```
class Workout:
    def __init__(self, start, end, cal=200):
        self.start, self.end = start, end
        self.calories = cal
    def get_start(self):
        return self.start
    def __str__(self):
        return f"<{self.calories} kcal>"

class RunWorkout(Workout):
    def __init__(self, start, end, elev=0):
        super().__init__(start, end)
        self.kind = "Running"
        self.elev = elev

rw = RunWorkout("9/30 1:35 PM", "9/30 3:35 PM",
elev=120)
print(rw.kind, rw.elev, rw.get_start())
print(rw)
```

Source: Lecture 12 · Inheritance & super()

Lec12 · Inheritance | Lec12：override、方法查找与子类替代

讲解

- 同名方法先找实例的类，找不到再沿父类向上，用第一个找到的。
- 子类写 get_calories 就覆盖父类；需要父类算法时用 super() 兜底。
- 父类 __str__ 里的 self.get_calories()，对子类实例会调子类版本。
- 能用父类的地方就能用子类；反过来不行，父类没有 get_elev。
- 易错：覆盖 get_calories 忘 return 会拿到 None；total_elevation([w1, rw]) 会 AttributeError。

代码与输出

```
class RunWorkout(Workout):
    def __init__(self, start, end, elev=0):
        super().__init__(start, end)
        self.kind = "Running"
        self.elev = elev
    def get_calories(self):
        base = super().get_calories()
        return base + self.elev * 2

def total_calories(ws):
    return sum(w.get_calories() for w in ws)

rw = RunWorkout("9/30 1:35 PM", "9/30 3:35 PM",
elev=120)
w1 = Workout("9/30 1:35 PM", "9/30 2:05 PM")
print(rw)                          # <440 kcal>
print(total_calories([w1, rw])) # 640
```

Source: Lecture 12 · Override & Method Lookup

PA3 起步指导

PA3 · Overview｜先读接口与限制，再决定数据结构

T1 · My Crafting｜30

- 实现 crafting(recipes, ingredients)。
- 原料顺序无关，但名称和数量必须精确匹配。
- 异常类型与“先验证全部配方”是接口的一部分。

T2 · Encoded Pairs｜30

- 统计二元 pattern 在编码展开后的出现次数。
- 展开长度可达 10^18，不能直接构造完整字符串。
- 利用编码的递归结构，重点思考拼接和重复边界。

T3 · File System｜40

- 模拟目录/文件树，支持 SIZE、PRINT、FIND、CLEAN。
- 递归遍历、路径定位、排序输出是核心。
- 失败操作输出 ERROR，且不能修改文件系统。

Due: 2026-11-05 21:00 · Source: SI100B_PA3.pdf

PA3 · Hints｜三题入手提示：只给方向，不代替你的设计

T1：先验证，再匹配

- 先逐条解析并检查所有 recipe：字段数、int 转换、数量 1–64。
- 把原料看作“名称 → 次数”的计数问题，而非位置匹配。
- 无匹配 raise LookupError；无效配方 raise ValueError。

T2：为每块保留摘要

- 单个字母是递归的基本情形。
- 合并两块时，除了内部答案，还要想跨界出现的位置。
- 重复 t 次应按份数和边界数计算，不要循环展开 t 次。

T3：树操作的公共步骤

- 先统一路径解析与节点查找，再实现各命令。
- SIZE/PRINT/FIND/CLEAN 可共享递归遍历思路。
- 打印/查找按要求排序；CLEAN 删除文件但保留目录。

Hints only; please design and implement your own solution

PA3 · Testing | 建议推进方式：小步实现，自建边界测试

实现与测试顺序

- 先抄写函数签名、返回类型、异常和输出格式，不急着写算法。
- T1 可运行 python test_crafting.py solution.py，但公开例子不够。
- T2 手算短编码，专门检查重叠 pattern 与跨块边界。
- T3 按 SIZE → PRINT → FIND → CLEAN 分步验证，再组合操作。
- 每完成一个小功能就测试；失败时先缩小到最小复现输入。

值得自测的边界（不是答案）

```
T1: 重复原料；前面已匹配但后面配方无效
T2: aa / 3[a]；1[a]1[b]；嵌套重复
T3: 空目录；不存在路径；目标是文件
    CLEAN 后再次 CLEAN；同级排序与缩进

# T1 只提交 solution.py，不要自行调用
```

Midterm 复习

Midterm · Official｜期中要求：以 Lecture 12 与后续官方通知为准

时间	地点	形式	范围	难度	总评占比
11/11 周三 3–4 节 11/13 周五 3–4 节 11/13 周五 11–12 节	信息学院机房 3-101、1A-109	Programming OJ 提交与评分	课堂 Python 排除 OOP 与 inheritance	低于作业 英文更简洁	41%
资料	**环境**	**网络/账号**	**证件与物品**	**个人安排**	**机房开放**
闭卷；一张 A4 cheat sheet；供草稿纸	Windows VS Code / PyCharm	仅 OJ 可联网 考前发新账号	带照片身份证件 电子设备与资料装包	场次/房间/座位 TBA 不同卷成绩对齐	第 9 周开放熟悉环境 具体时间 TBA

Source: Lecture 12, pp. 3–6

Midterm · Map｜2025 Rec7 的复习清单：这些都要在，不压缩

基础与程序流

- 数值类型、字符串、slice、I/O。
- 条件语句、while/for、range、break。
- 简单算法：guess-and-check、二分、Newton。

函数与容器

- 函数、return/None、作用域、默认参数。
- 高阶函数、lambda、tuple、*args。
- list 操作、mutability、alias/clone。

进阶与调试

- 列表推导式、异常、assert、调试。
- 字典、词频统计、items() 遍历。
- 递归、base case、调用栈。

Source: Fall 2025 Recitation 7, pp. 14–34; Lecture 0–10

Midterm · 数值 | 数值类型：float 是近似值，不是精确数

讲解要点

- int 精确；float 用二进制近似，0.1 累加十次会得到 0.999…，别用 == 判等。
- / 永远得 float；// 向下取整，不是截断；% 的符号跟除数走。
- int() 直接截掉小数；round() 是“四舍六入五成双”，round(2.5) 得 2。
- 浮点比较写 abs(a-b) < 1e-10；固定两位用 f"{x:.2f}"，只影响显示。
- 手推先算括号，再按“幂 → 乘除取余 → 加减”，不要从左往右瞎算。

例子

```
x = 0
for _ in range(10):
    x += 0.1
print(x == 1, abs(x - 1) < 1e-10)
print(int(3.9), round(2.5), -7 // 2, -7 % 2)
# -> False True
# -> 3 2 -4 1
```

Source: Rec7 p.17; Lecture 1 pp.38–42; Lecture 3 pp.55–57

Midterm · 字符串 | 字符串：不可变，但可以重新绑定

讲解要点

- 单双引号都行，要配对；+ 拼接、* 重复；int("3") 得 3，str(3) 得 "3"。
- 下标从 0 开始，-1 是最后一个；读 s[len(s)] 会 IndexError。
- 字符串不可变：s[0] = "X" 直接 TypeError；想改只能拼新串再绑回变量。
- 今年补充：strip() 去首尾空白，split() 切分，join() 拼回，in 判断子串。
- 回文、统计字符先想 s[::-1]、s.count("a")、in，再决定要不要写循环。

例子

```
s = " SI100B "
t = s.strip()
print(t[0], t[-1], t[1:4])
print("100" in t, t.lower())
# -> S B I10
# -> True si100b
```

Source: Rec7 p.18; Lecture 1 pp.51–58; Lecture 7 pp.19–20

Midterm · Slice｜切片：stop 不算，方向看 step

讲解要点

- s[start:stop:step]：含 start、不含 stop；step 为负表示从右往左。
- 省略写法：s[:3] 从头、s[3:] 到尾、s[:] 整个、s[::-1] 反转。
- 切片越界不报错：s[100:] 得空串，s[5:1] 也得空串；索引越界才报错。
- 手推顺序：先看 step 定方向，再找 start，最后在 stop 前停。
- 写完从两端各代一个下标验算，s[::2] 和 s[1::2] 要能一眼分开。

例子

```
s = "0123456"
print(s[1:3], s[1:6:2], s[::-1])
print(repr(s[10:]), repr(s[5:1]))
# -> 12 135 6543210
# -> '' ''
```

Source: Rec7 p.19; Lecture 1 pp.54–56

Midterm · I/O｜输入输出：OJ 只看你打印了什么

讲解要点

- input() 永远返回 str，要算数必须 int()/float()；OJ 上不要写提示语。
- print(a, b) 默认空格分隔、末尾换行；sep= 改分隔，end= 改结尾。
- f-string：f"{x:.2f}" 保留两位；比 "ans=" + str(x) 少一层括号。
- 一行多值用 a, b = map(int, input().split())；多行用循环读。
- 输出先看样例的空格、换行和末尾空格；print(*L) 和 join 都能打一行。

例子

```
a, b = map(int, input().split())
print(a + b, end="|")
print(f"{a / b:.2f}")

# 输入 3 4
# -> 7|0.75
```

Source: Rec7 p.20; Lecture 1 pp.61–73; Lecture 2 pp.4, 10–16

Midterm · 条件 | 条件：先判断，再分支，别把 = 当 ==

讲解要点

- = 是赋值，== 是比较；if x = 5 是 SyntaxError，写完先扫一眼。
- if/elif/else 从上往下，第一个为真的分支执行后其余跳过。
- and/or/not 有短路；可以用短路避免除零，但别写得太绕。
- 今年补充：0、""、[]、{}、None 都当假；1 <= x <= 10 可以连写。
- 写分支前先列情况，确认每个区间都覆盖、互不重叠，再翻译成代码。

例子

```
x = 5
if x % 2 == 1 and 0 < x < 10:
    print("odd")
elif x == 0:
    print("zero")
else:
    print("even")
# -> odd
```

Source: Rec7 p.21; Lecture 2 pp.18–29

Midterm · 循环 | 循环：range 少一位，break 只跳一层

讲解要点

- for x in 序列 直接取元素；range(start, stop, step) 不含 stop。
- while 适合“不知道几次”的场景，但循环体里必须改变量。
- break 只跳当前这一层；continue 只跳过本次剩余语句。
- 循环结束后循环变量还留着最后一个值；一次没进时可能没定义。
- 二分、牛顿属于“了解”层：知道每步怎么缩小、什么时候停即可。

例子

```
total = 0
for i in range(1, 6):
    if i == 4:
        break
    total += i
print(total, i)
# -> 6 4
```

Source: Rec7 p.22; Lecture 2 pp.34–48; Lecture 3 pp.8–9

Midterm · 函数 | 函数：return 把值交出去，print 只是给人看

讲解要点

- def 只是把函数建出来，不调用不执行；函数名后面没括号就是函数对象。
- return 立刻结束函数并把值交回调用处；没写 return 默认返回 None。
- print 是给人看的，return 是给程序用的；print(f()) 会先打印再拿到 None。
- 返回多个值其实是返回 tuple；默认参数放最后，别用可变对象做默认值。
- 读完函数先问：返回什么类型？每条分支都 return 了吗？

例子

```
def add(x, y):
    return x + y

def show(x, y):
    print(x + y)

print(add(1, 2))
print(show(1, 2))
# -> 3
# -> 3
# -> None
```

Source: Rec7 p.23; Lecture 4 pp.53–67; Lecture 5 pp.35–39, 48

Midterm · 作用域｜作用域：函数里的 x 不是外面的 x

讲解要点

- 每次调用都新建局部作用域，参数和函数内赋值的名字都在里面。
- 函数里能读外面的变量；但只要函数内赋值，Python 全程当它是局部变量。
- 先赋值再用会 UnboundLocalError；真要改全局得写 global x。
- 查找顺序是 LEGB：Local → Enclosing → Global → Built-in。
- 传 list 进函数能改内容，是别名，不是 global；画两个框手推。

例子

```
x = 5
def test():
    x = 1
    print(x)
test()
print(x)
# -> 1
# -> 5

# 函数内先 print(x)、后 x = 1
# -> UnboundLocalError
```

Source: Rec7 pp.24–25; Lecture 5 pp.55–71

Midterm · 高阶函数 | 高阶函数：函数也能当参数、当返回值

讲解要点

- 函数是对象：能赋给变量、放进 list/dict、当参数传、当返回值返回。
- f 和 f() 是两回事；传给 sorted 的 key 是函数对象，别写成 f()。
- map 返回迭代器，要 list(...) 才能打印或下标。
- 内层函数记住外层参数（闭包）；make_prod(2)(3) 才出 6。
- 看到 f(g, x) 先找 g 是哪个函数；看到 f(x)(y) 先算 f(x) 返回什么。

例子

```
def make_prod(a):
    def g(b):
        return a * b
    return g

double = make_prod(2)
print(double(3), make_prod(10)(3))
# -> 6 30
```

Source: Rec7 p.26; Lecture 6 pp.3–6, 32–33

Midterm · lambda | lambda：一次性小函数，别写语句

讲解要点

- 语法 lambda 参数: 表达式；表达式的结果就是返回值，不能写 return。
- 适合只用一次的小函数：排序 key、map/filter、立即调用。
- lambda x: x % 2 == 0 是函数对象；(lambda x: x + 1)(2) 才是调用。
- lambda 不能写类型注解，也不能跨多行；逻辑复杂就改回 def。
- 常和 sorted/max/min 的 key 一起考；先把 key 单独写出来，再压成一行。

例子

```
L = ["bbb", "a", "cc"]
print(sorted(L, key=lambda s: len(s)))
print((lambda x, y: x + y)(2, 3))
# -> ['a', 'cc', 'bbb']
# -> 5
```

Source: Rec7 p.27; Lecture 6 pp.48–50; Rec4「Lambda」

Midterm · tuple | tuple：逗号才是元组，不可变但能解包

讲解要点

- 圆括号不是关键，逗号才是：(1) 是 int，(1,) 才是单元素 tuple。
- 不可变：不能 t[0] = ...，但可以切片、拼接、len、max、in。
- 解包：a, b = b, a 靠 tuple 完成交换；return q, r 返回一个 tuple。
- *args 把多余位置参数打包成 tuple。
- tuple 里若放了 list，list 本身还能改，别把“不可变”理解成“里面全冻结”。

例子

```
t = (1, "a", 3)
print(t[1], t[::-1], t + (5,))
a, b = 1, 2
a, b = b, a
print(a, b)
# -> a (3, 'a', 1) (1, 'a', 3, 5)
# -> 2 1
```

Source: Rec7 p.28; Lecture 6 pp.64–73

Midterm · list | list：会改原列表的方法，大多返回 None

讲解要点

- list 可变：L[i] = x、切片赋值、append/extend/insert/pop/remove 都改原对象。
- sort/reverse/append 返回 None；sorted 才返回新列表。
- remove(x) 按值删第一个，del L[i]/pop(i) 按位置删。
- 今年补充：enumerate 取下标，zip 并排遍历，split/join 在字符串和列表间转换。
- 方法题先问：返回新对象还是改原对象？返回 None 的方法不要接返回值。

例子

```
L = [3, 1, 2]
x = L.sort()
print(L, x)
print(sorted(L, reverse=True))
# -> [1, 2, 3] None
# -> [3, 2, 1]
```

Source: Rec7 p.29; Lecture 7 pp.10–15, 22–29, 71–75

Midterm · alias | alias / clone：等号不是复制

讲解要点

- L2 = L 只是给同一个对象起第二个名字；改一个另一个跟着变。
- L[:]、L.copy()、copy.copy(L) 是浅拷贝：只复制最外层。
- 嵌套结构要 copy.deepcopy(L)；深拷贝成本高，但每层都是新对象。
- 函数传 list 参数也是别名：L.append(x) 会影响调用者，L = L + [x] 不会。
- 判断输出题先标 alias：哪些名字指向同一个 id，再按语句顺序改。

例子

```
L = [1, 2, 3]
L1 = L[:]
L2 = L
L.append(4)
print(L1, L2)
# -> [1, 2, 3] [1, 2, 3, 4]
```

Source: Rec7 p.30; Lecture 7 pp.66–70; Lecture 8 pp.3–78

Midterm · 推导式｜列表推导式：一行生成新列表

讲解要点

- [表达式 for 元素 in 可迭代对象 if 条件]：先取元素、再筛条件、最后算表达式。
- 它生成新列表，不改原列表；等价 for 循环要先建空列表再 append。
- 也能写字典推导式 {k: v for ...}；嵌套推导按 for 的书写顺序从外到内。
- 表达式里别塞 print、append 这类副作用；推导式是“造值”的。
- 不确定就先写等价 for 循环，再压成推导式，不容易把 if 的位置写错。

例子

```
L = [1, 2, 3, 4, 5]
print([e ** 2 for e in L if e % 2 == 1])
print({e: e ** 2 for e in range(3)})
# -> [1, 9, 25]
# -> {0: 0, 1: 1, 2: 4}
```

Source: Rec7 p.31; Lecture 8 pp.80–89

Midterm · 异常 | 异常与 assert：该停就停，该接就接

讲解要点

- try 里出错就跳到匹配的 except；没出错则跳过 except。
- else 只在无异常时执行；finally 无论怎样都执行。
- 捕获要写具体类型；裸 except 会把真 bug 一起吞掉。
- raise ValueError 主动报错；assert 条件不成立抛 AssertionError。
- 读 traceback 最后一行，再看调用栈；用最小输入复现。

例子

```
def avg(g):
    assert len(g) != 0, "no grades"
    return sum(g) / len(g)

try:
    print(avg([]))
except AssertionError as e:
    print("caught:", e)
# -> caught: no grades
```

Source: Rec7 p.32; Lecture 8 pp.96–106; Lecture 9 pp.3–16

Midterm · dict | dict：按键查，不按位置查

讲解要点

- dict 是 key → value 的映射；key 必须唯一且可哈希，list 不行。
- d[k] 缺键抛 KeyError；不确定就用 d.get(k, 默认值)。
- k in d 查键，不查值；查值要 v in d.values()。
- 增改 d[k] = v；删 del d[k]；items() 配 for k, v 最好用。
- 计数模板 freq[w] = freq.get(w, 0) + 1；遍历时别增删键。

例子

```
freq = {}
for ch in "abca":
    freq[ch] = freq.get(ch, 0) + 1
print(freq, freq["a"], "b" in freq, 1 in freq)
# -> {'a': 2, 'b': 1, 'c': 1} 2 True False
```

Source: Rec7 p.33; Lecture 9 pp.17–41; Lecture 10 pp.3–16

Midterm · 递归 | 递归：想清楚 base case，再想怎么缩小

讲解要点

- 三件事：base case 直接返回、递归步缩小成同类子问题、把子结果组合起来。
- 每次递归调用都有独立作用域；上一层在等下一层返回。
- 调用栈太深会 RecursionError；Python 默认约一千层。
- 返回类型要一致：base case 返回 0，递归步就别返回 list。
- 嵌套 list 典型场景：遇到 list 就递归，否则处理元素；先写终止条件。

例子

```
def fact(n):
    if n == 1:
        return 1
    return n * fact(n - 1)

print(fact(4))
# -> 24

def flatten(x):
    res = []
    for e in x:
        if isinstance(e, list):
            res.extend(flatten(e))
        else:
            res.append(e)
    return res
```

Source: Rec7 p.34; Lecture 10 pp.17–61

Midterm · 2026 补充｜今年课件反复强调的几件事

算法与边界

- 二分：low/high/guess 的更新与终止条件。
- Newton：初始值、epsilon、$0 < x < 1$ 的边界。
- guess-and-check：先有候选，再验证，最后停止。

函数与对象

- print 与 return 是两件事；没有 return 得到 None。
- LEGB：Local → Enclosing → Global → Built-in。
- 函数是对象；高阶函数、lambda、*args 要会手推。

容器与调试

- sort 改原表返回 None；sorted 返回新表。
- 浅复制只复制外层；嵌套结构用 deepcopy。
- dict 查找快；遍历时不要直接增删键。

Source: Lecture 1–10; 2025 Rec7 review map

Midterm · 易错诊所 1｜先看这三个坑：alias、排序、遍历删除

Alias / clone

- b = a 不是复制；改 b 就是改 a。
- a[:] 只复制外层，嵌套 list 仍共享。
- 需要完全独立时用 copy.deepcopy。

sort / sorted

- L.sort() 原地改并返回 None。
- sorted(L) 返回新列表，L 不变。
- 写 L = L.sort() 会把 L 变成 None。

遍历时删除

- for x in L 里边 remove，会跳过相邻元素。
- 先遍历 L[:] 副本，或先收集要删的下标。
- 同理，遍历 dict 时不要直接增删键。

Source: Lecture 7–8; Fall 2025 Rec7 pp. 23–33

Midterm · 易错诊所 2｜再看这三个坑：作用域、异常、递归

作用域

- 函数内先 print(x)、后 x = 1，会 UnboundLocalError。
- 局部变量不会自动读取同名全局变量。
- 需要修改全局变量时才用 global，别滥用。

异常控制流

- else 只在 try 正常结束时执行。
- finally 总会执行，但不会自动吞掉异常。
- except 要写具体类型；except: pass 会隐藏 bug。

递归

- 没有 base case 会 RecursionError。
- 参数没有靠近 base case，也会无限递归。
- 只调用不 return，当前层得到 None。

Source: Lecture 5, 8, 10; Fall 2025 Rec7 pp. 25–34

Midterm · 手推 1｜手推程序：先遮住答案，再逐行解释

① Alias

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

答案：**[1, 2, 3, 4]**

② sort 的返回值

```
L = [3, 1, 2]
x = L.sort()
print(L, x)
```

答案：**[1, 2, 3] None**

③ Slice / range

```
s = "0123456"
print(s[1:6:2])
print(s[::-1])
```

答案：**135**
6543210

④ Dict

```
d = {"a": 1}
print("a" in d, 1 in d)
print(d.get("b", 0))
```

答案：**True False**
0

Activity adapted from Fall 2025 Recitation 7 review style

Midterm · 手推 2 | 手推程序：作用域、异常、递归、复制

⑤ Scope

```
x = 5
def f():
    print(x)
    x = 1
try:
    f()
except Exception as e:
    print(type(e).__name__)
```

答案：**UnboundLocalError**

⑥ Exception

```
def f():
    try:
        return "try"
    finally:
        print("finally")
print(f())
```

答案：**finally**
try

⑦ Recursion

```
def f(n):
    if n == 0:
        return 1
    return n * f(n-1)
print(f(4))
```

答案：**24**

⑧ Copy

```
a = [1, [2]]
b = a[:]
a[1].append(3)
print(b)
```

答案：**[1, [2, 3]]**

Activity adapted from Fall 2025 Recitation 7 review style

Midterm · Tips｜2025 Rec7 的考前提醒，原样保留

读题与写代码

- 认真读题，胆大心细：Thinking Twice, Coding Once。
- 先写小例子，再写循环；边界先想空、单元素、极值。
- 自定义函数不要忘记 return；print 不能代替返回值。

调试

- 多 print 中间值，提交前删掉调试输出。
- 先读 traceback 最后一行的异常类型和原因。
- 循环注意 index、循环变量和终止条件。

复制与对象

- 传参时先分清 alias 还是 clone。
- 嵌套结构注意浅复制；需要独立副本用 deepcopy。
- list 可变，tuple 不可变；别把两者混用。

Source: Fall 2025 Recitation 7, pp. 35–36

Midterm · Plan｜最后阶段：用输出检验复习

一张 A4 怎么整理

- 放易混语法、常用方法、异常与递归模板。
- 记录自己错过的边界，不抄整份课件。
- 切片/排序/复制写成可快速查找的对照表。

练习顺序

- 先预测输出，再运行验证；错题隔天重做。
- 用最小输入测试空、单元素、重复、越界与失败路径。
- 读 traceback 最后一行，再沿调用栈定位。

考前与考场

- 按后续通知熟悉 Windows、OJ 与考场环境。
- 带照片身份证件和 A4；遵守离线及物品放置要求。
- 先读输入输出/return/raise 规格，不在 OJ 打印提示语。

复习以完整课件与官方范围为准；往年材料仅作练习参考

Thanks for Listening

Any questions?